

```
In [1]: import pandas as pd                #Step 1: Import Libraries
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep
```

```
In [2]: df = pd.read_csv('C:/Users/hp/Desktop/sample_fraud_detection.csv')
print(df.head())
print(df.info())
```

```
TransactionID  Amount  Time   V1    V2    V3  Class
0              1   100.5   500 -1.23  0.47  1.11     0
1              2   250.0  1500  0.35 -0.56 -0.78     0
2              3  1200.0  2000 -0.98  0.25 -0.13     1
3              4    75.0   300  0.12 -0.45  0.56     0
4              5   500.0  1000 -1.34  0.78 -0.89     1
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	TransactionID	5 non-null	int64
1	Amount	5 non-null	float64
2	Time	5 non-null	int64
3	V1	5 non-null	float64
4	V2	5 non-null	float64
5	V3	5 non-null	float64
6	Class	5 non-null	int64

```
dtypes: float64(4), int64(3)
```

```
memory usage: 412.0 bytes
```

```
None
```

```
In [3]: print(df.isnull().sum())        #Step 3: Check for Missing Values
```

```
TransactionID    0
Amount           0
Time             0
V1               0
V2               0
V3               0
Class            0
dtype: int64
```

```
In [4]: print(df['Class'].value_counts())    #Step 4: Basic Data Exploration
sns.countplot(x='Class', data=df)
plt.title('Class Distribution: 0=Legit, 1=Fraud')
plt.show()
```

```
Class
```

```
0    3
```

```
1    2
```

```
Name: count, dtype: int64
```



```
In [5]: plt.figure(figsize=(12,6))                                # Step 5: Visualize Feature Distribu
sns.histplot(df[df['Class']==0]['Amount'], color='green', label='Legit', kde=True)
sns.histplot(df[df['Class']==1]['Amount'], color='red', label='Fraud', kde=True)
plt.legend()
plt.title('Transaction Amount Distribution by Class')
plt.show()
```



```
In [7]: # Step 6: Prepare Data for Modeling
X = df.drop(['TransactionID', 'Class'], axis=1) # Features
y = df['Class']                                # Target
```

```
In [8]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
```

```
In [9]: model = LogisticRegression()           #Step 8: Train Logistic Regression Model
        model.fit(X_train, y_train)
```

```
Out[9]: LogisticRegression
```

```
LogisticRegression()
```

```
In [10]: y_pred = model.predict(X_test)       # Step 9: Predict on Test Data
```

```
In [11]: print("Accuracy:", accuracy_score(y_test, y_pred))      # Step 10: Evaluate Mode
        print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
        print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.5

Confusion Matrix:

```
[[1 0]
 [1 0]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

C:\Users\hp\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

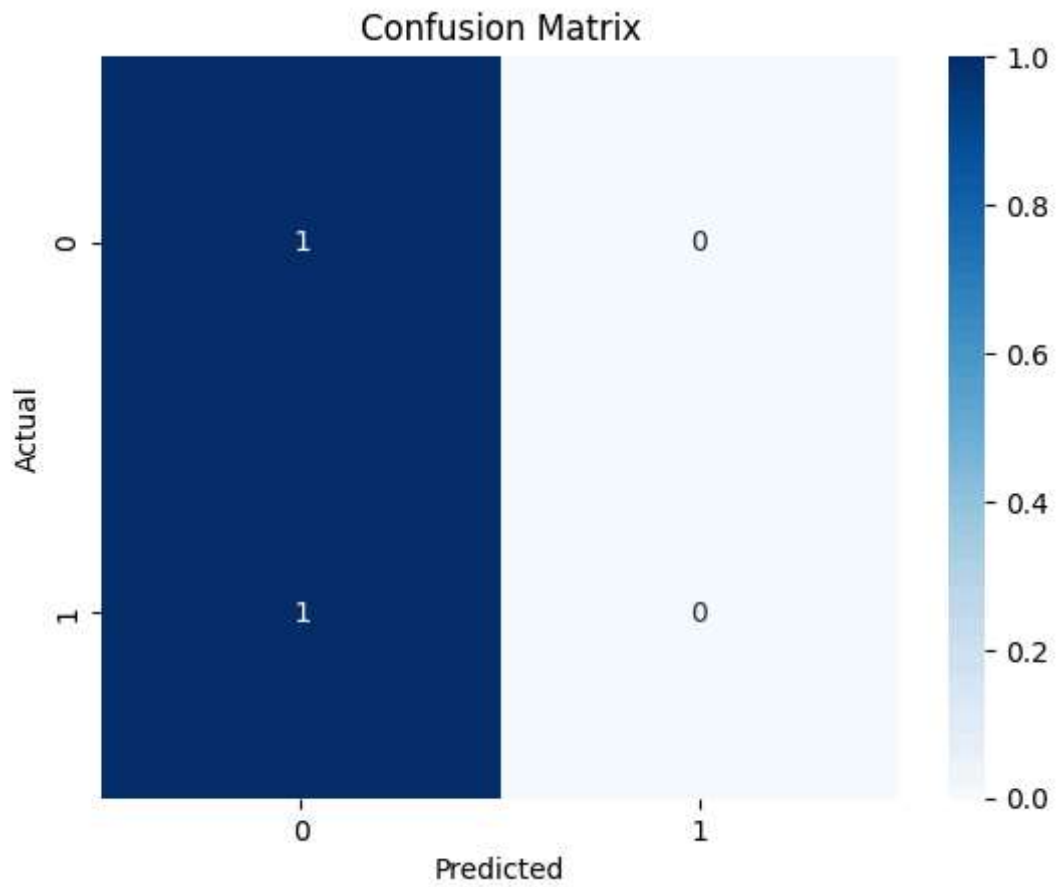
C:\Users\hp\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

C:\Users\hp\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```
In [12]: cm = confusion_matrix(y_test, y_pred)           # Step 11: Visualize Confusion Mat
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.title('Confusion Matrix')
        plt.show()
```



In []: